Reviewer Pack — Minimal Rank of Topological Quantum Field Theories over Bool…

Entry c824f42d21af · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:47:10 UTC

Minimal Rank of Topological Quantum Field Theories over Boolean Tensor Product Valuations

Entry ID: `c824f42d21af` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 13:47:10 UTC

1. Conjecture

Field A (mathematical branch): Topological Quantum Field Theory (TQFT) **Field B** (complexity object): Communication Complexity (Boolean Tensor Product)

Statement:

> ['For any given n-variables 3-CNF formula F, the minimal rank of its corresponding topological quantum field theory (TQFT) is asymptotically proportional to the width of the Boolean tensor product valuation of F.', 'More formally, for all fixed ε > 0 and sufficiently large n, there exist constants c_1, c_2 such that c_1 * log(n) ≤ min_rank(TQFT(F)) ≤ c_2 * val_width(F), where min_rank(TQFT(F)) is the minimal rank of the TQFT associated with F and val_width(F) is the width of the Boolean tensor product valuation of F.', 'The constant c_1 cannot be improved to o(log(n)), unless NP = P.']

Rationale (proposer's reasoning):

> ['Topological quantum field theories (TQFTs) are known to encode complex topological information in a manner that is invariant under certain transformations. By linking TQFTs with the Boolean tensor product valuation, we may uncover a novel way of characterizing the complexity of computational problems.', 'The use of TQFTs in complexity theory has been limited, and their potential role in communication complexity is an intriguing direction for research. This conjecture proposes a concrete link between these two fields, which could potentially lead to new insights into both topological quantum physics and computational complexity.']

Taxonomy category: `TQFTBOOLEANVAL` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `751f9170b8be3e52`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The ratio of log(min_rank(TQFT(F))) to log(val_width(F)) for each 3-CNF formula F is within a constant factor.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([1, -1]) * random.randint(1, n) for _ in range(3)]
            if len(set(clause)) == 3:
                clauses.append(clause)
        return clauses

    def boolean_tensor_product_width(clauses):
        width = 0
        for clause in clauses:
            width |= sum(abs(x) for x in clause)
        return width

    def min_rank_tqft(clauses):
        # Placeholder function to simulate TQFT computation
        # Actual implementation would depend on the specific TQFT construction
        rank = len(clauses) * 2  # Simplified example
        return rank

    n = random.randint(5, 40)
    formula = generate_random_3cnf(n)
    val_width = boolean_tensor_product_width(formula)
    min_rank = min_rank_tqft(formula)

    if val_width == 0:
        return {
```

```
            "metric_name": "log_ratio",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "val_width=0"
        }

    log_ratio = math.log(min_rank) / math.log(val_width)
    conjecture_holds = log_ratio >= 1 and log_ratio <= 2

    return {
        "metric_name": "log_ratio",
        "metric_value": log_ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(seed) for seed in sys.argv[1:]]
    else:
        seeds = [2**i + 3 for i in range(5, 8)]  # Default list of 30 primes

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        result_type = "SUPPORTED"
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        result_type = "FALSIFIED"

    print(f"RESULT: {result_type} mean={mean_value} std=0 support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
636195871675864, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 1.0184336218414725, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 0.9800412679514446, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 0.9254704927837457, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 0.9464885226021431, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 1.0751152425431807, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 0.9703601998932891, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 1.0287807908244249, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 0.9715715168743821, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 1.0171476859204753, 'instances_tested': 1, 'conj
```

```
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 1.0452793901921185, 'instances_tested': 1, 'conj
TRIAL: {'metric_name': 'log_ratio', 'metric_value': 1.0038010728610447, 'instances_tested': 1, 'conj

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25c88973.py", line 86, in <module>
    print(f"RESULT: {result_type} mean={mean_value} std=0 support_fraction={support_fraction}")
                                          ^^^^^^^^^^
NameError: name 'mean_value' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14191
2	preregistration	ollama_remote	glm4:latest	0	0	5833
3	novelty	ollama_remote	glm4:latest	0	0	4832
4	novelty	ollama_remote	glm4:latest	0	0	5223
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15139
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8547
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7396
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7197
9	judge	ollama_remote	glm4:latest	0	0	12023

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 80381 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c824f42d21af.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c824f42d21af.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c824f42d21af.tar.gz` (if generated)